

JUNIA · ISEN BORDEAUX 4^{ème} année
Module Bases de données avancées et Big Data

ÉVALUATION FINALE

Architecture Data 360°

Spark · MongoDB · Cassandra · (Bonus : Neo4j)

Durée
4 heuresModalité
Travail en binômeBarème
25 pts + 3 pts bonus

Barème récapitulatif

Partie	Contenu évalué	Points
Partie 1	Architecture Docker	4 pts
Partie 2	Analyse de logs avec Spark	6 pts
Partie 3	Base e-commerce MongoDB	5 pts
Partie 4	Modélisation Cassandra	5 pts
Partie 5	Corrélation Sécurité ↔ E-commerce	5 pts
Bonus	Neo4j – Graphe de fraude	+3 pts
	TOTAL	25 pts

Contexte du projet

Vous intervenez en tant qu'équipe Data / Architecture pour une plateforme e-commerce.

L'entreprise souhaite :

- Détecter des comportements suspects à partir de logs Apache.
- Consolider les données e-commerce (commandes).
- Produire un reporting optimisé pour certaines requêtes métier.
- Identifier d'éventuelles fraudes : IP suspectes correspondant à des clients ayant effectué des commandes.

Vous devez concevoir une architecture conteneurisée complète et exploitable. Aucun docker-compose complet ne vous est fourni — vous devez reconstruire la stack à partir des briques vues durant les TP.

Datasets fournis

Trois fichiers vous sont fournis. Ils doivent être placés dans le dossier local :

```
./data
```

Fichier	Format	Contenu / Usage
access.log	Apache Combined Log	Logs serveur web — analysé via Spark (Partie 2)
clients.json	JSON Array	60 clients avec ip_address — importé dans MongoDB (Partie 3)
commandes.json	JSON Array imbriqué	~210 commandes avec tableau items — importé dans MongoDB (Partie 3)

PARTIE 1 — Architecture Docker (4 pts)

Objectif

Construire un fichier docker-compose.yml unique contenant l'ensemble de la stack nécessaire au projet.

Services attendus

- Un service Spark Master + Spark Worker
- Un MongoDB dédié aux logs
- Un MongoDB dédié à l'e-commerce
- Un service Cassandra
- (Bonus) Un service Neo4j

Contraintes techniques

Réseau	Un seul réseau commun à tous les services
Persistance	Volumes nommés pour chaque base de données
Ports	Ports cohérents et non conflictuels
Nommage	Noms de service explicites et lisibles
Originalité	Pas de copier-coller intégral d'un ancien docker-compose

Montages de volumes exigés

Dans le service MongoDB :

```
volumes:
  - mongo_data:/data/db
  - ./data:/datasets:ro
```

Dans le service Jupyter :

```
volumes:
  - ./data:/home/jovyan/work/data:ro
```

Objectif

/datasets → utilisé pour importer les JSON dans MongoDB via mongoimport
/home/jovyan/work/data → utilisé par Spark pour lire access.log

Livrable attendu

Fichier

docker-compose.yml fonctionnel permettant le démarrage complet via docker-compose up et la connexion aux bases depuis vos notebooks ou scripts.

PARTIE 2 — Analyse de Logs avec Spark (6 pts)

Fichier source

Chemin	/home/jovyan/work/data/access.log
--------	-----------------------------------

Un fichier de logs Apache au format Combined Log Format.

Étapes à réaliser

1. Lecture et parsing des logs

Lire le fichier de logs avec Spark et extraire les champs suivants :

- ip — adresse IP source
- timestamp — date et heure de la requête
- méthode — verbe HTTP (GET, POST, ...)
- url — chemin de la ressource
- status_code — code de retour HTTP

2. Enrichissement des données

Calculer et ajouter les colonnes suivantes :

- status_group : regroupement du code HTTP (2xx / 4xx / 5xx)
- is_error : booléen vrai si le code est ≥ 400
- hour : heure extraite du timestamp

3. Profil par IP

Construire un profil agrégé par adresse IP contenant :

Champ	Description
<i>nb_requetes</i>	Nombre total de requêtes émises par l'IP
<i>nb_erreurs</i>	Nombre de requêtes ayant généré une erreur
<i>taux_erreur</i>	Ratio <i>nb_erreurs</i> / <i>nb_requetes</i>
<i>nb_requetes_nuit</i>	Nombre de requêtes entre 01h et 04h

4. Stockage dans MongoDB

Paramètre	Valeur
Base de données	logs_db
Collection	ip_profiles
URI de connexion	mongodb://tp_mongodb:27017

Rappel pédagogique : vous devez réutiliser les techniques vues dans les notebooks précédents — DataFrame, groupBy, calculs dérivés, écriture vers MongoDB.

PARTIE 3 — Base E-commerce MongoDB (5 pts)

Données fournies

Les fichiers clients.json et commandes.json sont des JSON Array. L'import doit être réalisé depuis le conteneur MongoDB.

Procédure d'import

1. Accéder au shell du conteneur

```
docker exec -it tp_mongodb bash
```

2. Importer les deux collections

```
var fs = require("fs")
var content = fs.readFileSync("/datasets/clients.json", "utf8")
var data = JSON.parse(content)
db.customers.insertMany(data)
```

3. Vérifier l'import

```
mongosh
use ecommerce_db
db.customers.countDocuments()
```

L'import doit être fonctionnel et vérifié avant de passer à la suite. Les deux collections doivent renvoyer un nombre de documents cohérent.

Travail demandé

1. Insertion des données

Insérer les données dans MongoDB dans la base ecommerce_db.

2. Pipeline d'agrégation

Écrire un pipeline permettant de calculer :

- Total dépensé par client
- Panier moyen par client
- Top 3 des produits par quantité vendue

3. Format de sortie

Les résultats doivent être produits sous forme JSON.

Opérateurs MongoDB requis

Opérateur	Usage attendu
<i>\$unwind</i>	Décomposer le tableau des items d'une commande
<i>\$group</i>	Regrouper par client ou par produit
<i>\$multiply</i>	Calculer le sous-total (quantité × prix unitaire)

PARTIE 4 — Modélisation Cassandra (5 pts)

Objectif

Construire un keyspace ecommerce_reporting permettant d'optimiser deux requêtes métier critiques selon le principe du Query-Driven Design.

Requêtes à optimiser

#	Requête métier	Exigence
Q1	Historique des commandes d'un client	Tri décroissant par date
Q2	Liste des commandes d'un jour donné	Filtrage par date uniquement

Étapes à réaliser

- Créer le keyspace ecommerce_reporting
- Créer les tables adaptées avec des PRIMARY KEY cohérentes
- Insérer les données (écriture dupliquée si nécessaire)
- Exécuter les requêtes demandées et présenter les résultats

Justification

Vous devez justifier brièvement (5 lignes max) votre choix de partition key et de clustering column pour chaque table. Cette justification fera partie de votre README.txt.

PARTIE 5 — Corrélation Sécurité ↔ E-commerce (5 pts)

Critères de détection d'une IP suspecte

Indicateur	Seuil de suspicion
Volume de requêtes	<i>nb_requetes > 100</i>
Taux d'erreur	<i>taux_erreur > 0.30 (plus de 30% d'erreurs)</i>
Activité nocturne	<i>Présence entre 01h et 04h</i>

Travail demandé

- Interroger la collection ip_profiles dans logs_db pour identifier les IP suspectes (les trois critères doivent être vérifiés simultanément)
- Croiser ces IP avec les données clients de ecommerce_db pour détecter d'éventuels clients ayant passé des commandes
- Générer un rapport JSON final structuré

Structure du rapport JSON attendu

```
{
  "ip": "192.168.1.42",
  "nb_requetes": 312,
  "taux_erreur": 0.47,
  "nb_requetes_nuit": 89,
  "client_id": "C_00145",
  "total_depense": 1247.80
}
```

Livrable principal

Ce rapport JSON constitue votre livrable principal et est évalué sur la complétude, la justesse des données et la cohérence du croisement.

BONUS — Neo4j : Graphe de Fraude (+3 pts)

Modèle de graphe à construire

(Customer) -[:PASSE]-> (Order) -[:CONTIENT]-> (Product)

Requête à implémenter

Écrire une requête Cypher permettant de répondre à la question suivante :

"Quels clients ont acheté des produits en commun avec un client suspect ?"

Livrables à rendre

	Fichier / Document	Contenu attendu
1	docker-compose.yml	Stack complète et fonctionnelle
2	Notebook(s) ou scripts Python	Parties 2, 3 et 5 — code commenté
3	Script Cassandra (.cql)	Création du keyspace, tables, insertions et requêtes
4	Rapport JSON final	Résultat de la corrélation Sécurité ↔ E-commerce
5	README.txt	Répartition des tâches et justification des choix techniques

Contenu du README.txt

- Répartition claire des tâches entre les deux membres du binôme
- Justification des choix techniques pour l'architecture Docker
- Justification des partition keys et clustering columns Cassandra
- Tout commentaire utile sur les difficultés rencontrées

Conseils et rappels

Gestion du temps conseillée

- Partie 1 — Docker : 35 min
- Partie 2 — Spark : 55 min
- Partie 3 — MongoDB : 55 min
- Partie 4 — Cassandra : 35 min
- Partie 5 — Corrélation : 35 min
- Bonus Neo4j : si temps disponible

Rappels importants :

- Prenez le temps de tester chaque service avant de passer à la partie suivante.
- Commentez votre code — la lisibilité fait partie de l'évaluation.
- En cas de blocage sur une partie, passez à la suivante et revenez-y si le temps le permet.
- La justification de vos choix (README, Cassandra) est valorisée autant que le code lui-même.
- Le bonus Neo4j n'est traité qu'une fois les parties obligatoires terminées.